
Formal Synthesis of Certifiably Robust Neural Lyapunov-Barrier Certificates

Chengxiao Wang
University of Illinois,
Urbana-Champaign
cw124@illinois.edu

Haoze Wu
Amherst College
hwu@amherst.edu

Gagandeep Singh
University of Illinois,
Urbana-Champaign
ggnds@illinois.edu

Abstract

Neural Lyapunov and barrier certificates have recently been used as powerful tools for verifying the safety and stability properties of deep reinforcement learning (RL) controllers. However, existing methods offer guarantees only under fixed ideal unperturbed dynamics, limiting their reliability in real-world applications where dynamics may deviate due to uncertainties. In this work, we study the problem of synthesizing *robust neural Lyapunov barrier certificates* that maintain their guarantees under perturbations in system dynamics. We formally define a robust Lyapunov barrier function and specify sufficient conditions based on Lipschitz continuity that ensure robustness against bounded perturbations. We propose practical training objectives that enforce these conditions via adversarial training, Lipschitz neighborhood bound, and global Lipschitz regularization. We validate our approach in two practically relevant environments, Inverted Pendulum and 2D Docking. The former is a widely studied benchmark, while the latter is a safety-critical task in autonomous systems. We show that our methods significantly improve both certified robustness bounds (up to 4.6 times) and empirical success rates under strong perturbations (up to 2.4 times) compared to the baseline. Our results demonstrate effectiveness of training robust neural certificates for safe RL under perturbations in dynamics.

1 Introduction

Deep Reinforcement learning (RL) has demonstrated remarkable performance in a wide range of sequential decision-making tasks, such as robotic control [16], autonomous driving [30], game playing [25]. However, using RL controllers in safety-critical domains such as autonomous driving remains challenging due to the lack of formal guarantees on safety and stability. Classical control theory provides formal guarantees through analytic Lyapunov and barrier functions, but deriving such certificates for the complex dynamics of modern DRL tasks is notoriously difficult. To sidestep this bottleneck, recent work has explored parameterizing certificates with neural networks. These methods learn a neural certificate function jointly with the policy, and prove the validity of the learned certificate with formal verifiers [1, 2, 6, 22].

By and large, the validity of the neural certificate is only established for a fixed ideal dynamics in existing work. However, in practice, the actual observed next state by the controller might deviate from the ideal dynamics, due to modeling errors [28], actuation noise [27], or environmental perturbations [23, 28]. The existence of these unavoidable uncertainty means that the controller may violate the safety or stability properties even if it has been fully verified against the ideal dynamics. Indeed, such sim-to-real-error has long been recognized and methods for guaranteeing robustness against bounded disturbances have been studied for analytic Lyapunov certificates [5, 11, 12]. However, analogous methods that guarantee such robustness property for *neural* certificates are still largely unexplored.

In this work, we bridge this gap and study the problem of synthesizing neural certificates robust against norm-bounded time-varying uncertainty. Unlike existing approach that obtains probabilistic guarantees for stochastic systems [17, 44], we aim to give deterministic guarantee against bounded perturbations in the system dynamics. We formally define a robust neural Lyapunov-barrier certificate and develop a training framework to obtain such certificates. The key idea is to enforce a stronger decrease condition on the Lyapunov function: instead of requiring decrease only at the next state, we ensure that the Lyapunov value decreases across a neighborhood of perturbed next states, so that it keeps the safety and liveness property even under perturbation. We formally establish the relationship between the Lipschitz constant of the certificate and the strengthened decrease condition, and, based on this theoretical result, investigate several candidate loss functions for training the robust certificate. We evaluated our approach in two case studies, and demonstrated that our robust training approaches, including adversarial training, and Lipschitz regularization, improve both certified robustness bound and empirical success rate under perturbations, by up to 4.6 times and 2.4 times over the baseline method respectively.

To summarize, our contributions include:

- We study, for the first time, the problem of synthesizing neural Lyapunov/barrier certificates that provably satisfy the **robust Lyapunov condition**, which guarantees the Lyapunov decrease condition under bounded perturbations in the system dynamic.
- We establish sufficient theoretical conditions for robustness that leverage Lipschitz continuity, and develop multiple practical training methods to enforce robustness, including global Lipschitz regularization, local neighborhood bounds, and adversarial training.
- We demonstrate the practical benefits of our approach in two case studies. Our approach improves the certified perturbation bounds by up to 4.6 times, and for empirical robustness, we achieve up to 2.4 times success rate compared to baselines under strong perturbations.

2 Related Work

In this section, we review related work on control certificates. A detailed discussion of other related work can be found in Appendix A. There are extensive works on safe control, most of which using Lyapunov certificate to guarantee the liveness and use barrier function to guarantee the safety, with traditional approaches including numerical methods, polynomial optimization, and simulation-guided synthesis [13]. Recent work has explored jointly training neural controllers and certificates using sampled data [10, 20], and verifying trained certificates with formal tools [1, 2, 6, 22, 24]. However, they only consider the dynamics that are deterministic without any perturbations. There has also been work on generating neural certificate for stochastic control system [17, 44], which provides probabilistic reach-avoid guarantees. In contrast, our work provides deterministic reach-avoid guarantee under norm-bounded time-varying perturbations. Previous work in neural certificate training has also considered enforcing Lipschitz continuity as a regularization term [10, 17, 44] in order to improving the efficiency and the training and verification. In contrast, we leverage Lipschitz continuity to improve the robustness of the certificate against perturbations in the system dynamics.

3 Background

3.1 Notation

We denote the l_p norm of a vector x as $\|x\|_p$, the spectral norm of matrix A as $\|A\|_2$. The l_p ball of radius δ around x is $\mathcal{B}_{\delta,p}(x) = \{x' : \|x' - x\|_p \leq \delta\}$. $\mathcal{X}$ is the set of possible states and $\mathcal{U}$ is the set of possible control inputs. The function $f : \mathcal{X} \times \mathcal{U} \rightarrow \mathcal{X}$ is a transition function that defines the system's transition dynamics. Given a current state x and a control input u , the transition function outputs the next state. A policy $\pi : \mathcal{X} \rightarrow \mathcal{U}$ maps each state to a control action and $u = \pi(x)$ is the control action taken by policy π at state x .

3.2 Reach-While-Avoid task

We consider a discrete-time dynamical system, which can be either linear or non-linear, defined as:

$$x_{t+1} = f(x_t, u_t), \quad x_t \in \mathcal{X} \subset \mathbb{R}^n, \quad u_t \in \mathcal{U} \subset \mathbb{R}^m, \quad (1)$$

where x_t denotes the system state at time t , u_t is the control input. We define three important subsets of the state space: the **unsafe set** $\mathcal{X}_U \subset \mathcal{X}$, which must be avoided; the **goal set** $\mathcal{X}_G \subset \mathcal{X} \setminus \mathcal{X}_U$, which the system must eventually reach; and the **initial set** $\mathcal{X}_I \subset \mathcal{X} \setminus \mathcal{X}_U$, where the system starts.

The objective of *Reach-While-Avoid* (RWA) task is to synthesize a policy π such that, for every $x_0 \in \mathcal{X}_I$, the induced trajectory $\{x_t\}_{t \geq 0}$ satisfies:

$$\exists \tau \geq 0, x_\tau \in \mathcal{X}_G, \quad \wedge \quad \forall t \in [0, \tau], x_t \notin \mathcal{X}_U.$$

3.3 Control Lyapunov Barrier Function

A *Control Lyapunov function* (CLF) represents the energy level of a state. As time passes, energy decreases until the system reaches the zero-energy equilibrium point [15]. A *Control Barrier function* (CBF) [3], also an energy-based function, is used to certify that the system will never enter an unsafe state. This is done by assigning unsafe states' function values above a threshold and proving that the system's function value will never exceed the threshold.

A function $V : \mathcal{X} \rightarrow \mathbb{R}$ is a *Control Lyapunov Barrier Function* (CLBF) if it combines the properties of CLF and CBF. More formally, it can be defined as follows:

Definition 3.1 (Control Lyapunov Barrier Function). *A function $V : \mathcal{X} \rightarrow \mathbb{R}_{\geq c}$ is a Control Lyapunov Barrier Function with parameters $(\alpha, \beta, \epsilon)$ w.r.t. sets $\mathcal{X}_I$ (initial), $\mathcal{X}_G$ (goal), and $\mathcal{X}_U$ (unsafe), if $\alpha > \beta > c$, $\epsilon > 0$ and the following conditions hold:*

$$V(x) \leq \beta, \quad \forall x \in \mathcal{X}_I, \quad (2)$$

$$V(x) - V(f(x, \pi(x))) \geq \epsilon, \quad \forall x \in \{x \in \mathcal{X} \setminus \mathcal{X}_G \mid V(x) \leq \beta\}, \quad (3)$$

$$V(x) \geq \alpha, \quad \forall x \in \mathcal{X}_U. \quad (4)$$

Intuitively, this definition encodes both safety and liveness properties. For safety property, since Eq. (2) ensures that trajectories starting within a safe region and Eq. (3) requires the certificate value to decrease at each step, the trajectory never enters the unsafe region defined by Eq. (4). For liveness property (i.e., the reachability of the goal state), although no explicit constraint is added in $V(x)$ for states in $\mathcal{X}_G$, it is still guaranteed because without entering $\mathcal{X}_G$, the certificate value would decrease infinitely by Eq. (3), contradicting the fact that V is lower-bounded.

Naturally, this concept can be applied to the RWA task, where the sets $\mathcal{X}_I$, $\mathcal{X}_G$, and $\mathcal{X}_U$ represent the initial, goal, and unsafe regions of the task, respectively. In this context, the CLBF for the RWA task is referred to as the *Lyapunov Barrier Certificate* for the specific RWA task. These constraints ensure that any trajectory $\{x_t\}_{t \geq 0}$ starting from $x_0 \in \mathcal{X}_I$ given by Eq. (1) stay away from $\mathcal{X}_U$, and target toward $\mathcal{X}_G$, while monotonically decreasing the certificate function V .

3.4 Global Lipschitz Bound

Consider a feedforward neural network with ℓ layers, mapping input x to output $x_{\ell+1}$, defined by:

$$x_1 = x, \quad x_{k+1} = \phi(W_k x_k + b_k) \text{ for } k = 1, \dots, \ell - 1, \quad x_{\ell+1} = W_\ell x_\ell + b_\ell,$$

where $W_k \in \mathbb{R}^{n_{k+1} \times n_k}$, $b_k \in \mathbb{R}^{n_{k+1}}$ are learned parameters, and ϕ is the element-wise activation function. A global Lipschitz constant L_p for a function $f : \mathbb{R}^{n_1} \rightarrow \mathbb{R}^{n_{\ell+1}}$ with respect to a norm $\|\cdot\|_p$ is defined as:

$$L_p = \sup_{x, y \in \mathbb{R}^{n_1}, x \neq y} \frac{\|f(x) - f(y)\|_p}{\|x - y\|_p}$$

The function is said to be Lipschitz continuous if L_p exists and is finite. For activation functions with Lipschitz constant equal to 1, the global Lipschitz bound of the network with respect to the l_2 -norm can be upper-bounded by the product of the spectral norms of the weight matrix of each layer [34]:

$$L_2 \leq \prod_{k=1}^{\ell} \|W_k\|_2 \quad (5)$$

While this work focuses on the most common ReLU activations, our approach can extend to other popular activation functions that are 1-Lipschitz (e.g., Leaky ReLU, SoftPlus, Tanh, Sigmoid, ArcTan, or Softsign), as the same spectral norm-based Lipschitz bound applies.



(a) Standard CLBF-certified controller. The nominal trajectory reaches the goal X_G , but after perturbations are added to each state, it may violate safety and enter the unsafe region X_U .

(b) Robust CLBF-certified controller. At each step, the controller considers a full set of reachable next states within a perturbation neighborhood. All perturbed trajectories reach the goal region X_G .

Figure 1: Comparison of standard vs. robust certified controllers under bounded state perturbations.

Extension to General Norms: Following the work in [35] we can upper bound the Lipschitz constant L_p with respect to l_p norm, by a scaled version of the l_2 -Lipschitz constant:

$$L_p \leq K_{p,2} \cdot L_2, \quad (6)$$

Specifically, $K_{p,2} = n^{(\frac{1}{2}-\frac{1}{p})}m^{(\frac{1}{p}-\frac{1}{2})}$ where n and m are the input and output dimensions of the neural network. The detailed derivation is provided in Appendix B.1.

4 Methodology

In Section 4.1, we introduce a robust extension of the standard CLBF condition, requiring the certificate to decrease not only at the next state but over its neighborhood, ensuring robustness to perturbations. In practice, this accounts for errors in the environment’s computed state. As reachable sets may grow exponentially, we require all to remain outside the unsafe region. Section 4.2 presents three loss function designs supporting the robust CLBF condition: an adversarial loss for empirical robustness, a local robustness loss using global Lipschitz bounds to conservatively upper-bound certificate values within a perturbed neighborhood, and a global Lipschitz regularization for certifiable robustness over the full state space. The overall training algorithm follows a counterexample-guided inductive synthesis (CEGIS) loop [32]. A full description of the algorithm is provided in Appendix D.

4.1 Robust Lyapunov Barrier Certificate

We first define what it means for a controller of an RWA task to be robust under state perturbations. Figure 1 provides an illustrative example. In an RWA task, the robust controller is required to reach the goal region $\mathcal{X}_G$ from an initial state in $\mathcal{X}_I$, while avoiding the unsafe region $\mathcal{X}_U$, even when states computed by the environment at each time step are perturbed by a bounded amount. The definition below formalizes this requirement.

Definition 4.1 (Robust Controller for Reach-While-Avoid (RWA) Task). *A controller for Reach-While-Avoid task is said to be robust under state perturbations $\delta \in \mathbb{R}$ in the l_p norm if, for any initial state $x_0 \in \mathcal{X}_I$, any induced trajectory $\{x_t\}_{t=0}^T$ given by $x_{t+1} = f(x_t, u_t) + \delta_t$ where f is the transition function, u_t is the control input, $\|\delta_t\|_p \leq \delta$, satisfies:*

$$x_0 \in \mathcal{X}_I, \quad \wedge \quad \exists \tau \geq 0, \quad x_\tau \in \mathcal{X}_G, \quad \wedge \quad \forall t \in [0, \tau - 2], \quad \mathcal{B}_{\delta,p}(f(x_t, u_t)) \cap \mathcal{X}_U = \emptyset.$$

Unlike traditional controllers for RWA tasks, our robust controller definition requires that, starting from any state reachable from the initial state (accounting for perturbations), the set of all possible next states must remain within the safe region. Although the controller output at each step determines a specific next state, we consider the entire neighborhood around it that state perturbations might induce. At each subsequent step, any state within the current reachable set may transition into another perturbed region, leading to a state expansion.

To certify the controller satisfies this definition, we extend the classical Lyapunov Barrier Certificate to account for perturbations in states. Specifically, we modify the decrease condition Eq. (3) to ensure

that the certificate value decreases from x_t to x_{t+1} even under perturbation in x_{t+1} . More formally, we define the *robust Lyapunov Barrier Certificate* as follows:

Definition 4.2 (Robust Lyapunov Barrier Certificate for RWA tasks). *A function $V : \mathcal{X} \rightarrow \mathbb{R}_{\geq c}$ is a (δ, p) -Robust Lyapunov Barrier Certificate with parameters $(\alpha, \beta, \epsilon)$ for a RWA task w.r.t. sets $\mathcal{X}_I$ (initial), $\mathcal{X}_G$ (goal), and $\mathcal{X}_U$ (unsafe), if $\alpha > \beta > c$, $\epsilon > 0$ and the following conditions hold:*

$$V(x) \leq \beta, \quad \forall x \in \mathcal{X}_I, \quad (7)$$

$$V(x) - V(y) \geq \epsilon, \quad \forall x \in \{x \in \mathcal{X} \setminus \mathcal{X}_G \mid V(x) \leq \beta\}, \forall y \in \mathcal{B}_{\delta,p}(f(x, \pi(x))) \quad (8)$$

$$V(x) \geq \alpha, \quad \forall x \in \mathcal{X}_U. \quad (9)$$

This robust version strengthens the classical certificate by requiring the decrease condition to hold for all next states within a perturbation ball. Intuitively, the controller must still drive the system toward the goal and avoid unsafe regions despite small perturbations. The value function V is therefore robustly decreasing until the goal is reached and remains outside unsafe states. With this strengthened certificate condition, we can formally guarantee robust success of the trajectory:

Theorem 4.1 (Robust RWA Guarantee). *Let $\{x_t\}_{t=0}^{\infty}$ be a trajectory generated by policy π and transition function f , and the perturbation at each state is less than δ under l_p norm, i.e., $x_{t+1} \in \mathcal{B}_{\delta,p}(f(x_t, \pi(x_t)))$. If there exists a (δ, p) -Robust Lyapunov Barrier Certificate $V : \mathcal{X}_s \rightarrow \mathbb{R}_{\geq c}$ where $\mathcal{X}_s = \{x_t \mid \forall t \in [0, \tau - 1]. x_t \notin \mathcal{X}_G\} \subset \mathcal{X}$ with parameters $(\alpha, \beta, \epsilon)$, satisfies condition Eq. (7)–(9), then the trajectory: (i) will reach the goal region $\mathcal{X}_G$ in finite time and (ii) will never enter the unsafe region $\mathcal{X}_U$, thereby completing the robust RWA task under state-wise perturbations.*

The detailed proof is provided in Appendix B.2. Here $\mathcal{X}_s$ denotes the set of all states in the trajectory before reaching the goal. At a high level, the guarantee follows from the fact that the robust Lyapunov condition ensures a decrease of V under bounded perturbations at each step. If any perturbed next state enters the unsafe set $\mathcal{X}_U$, it would violate the safety condition enforced by the certificate Eq. (9). Similarly, if the trajectory failed to reach the target, $V(x_t)$ would decrease indefinitely, contradicting the lower bound c of the certificate. Hence, satisfying the robust certificate conditions along the trajectory implies that the RWA task is completed under perturbations on states.

4.2 Training Objective

To synthesize a controller and certificate that satisfy the robust CLBF conditions under norm-bounded perturbations, we design a training objective with multiple loss terms. In this section, we describe how these loss components are formulated and how they relate to the robust CLBF condition. The total loss combines these terms and is later used in our training algorithm described in Appendix D.

Initialization Loss: This loss term proposed in [22] enforces $V(x) \leq \beta$ for initial states $x \in \mathcal{X}_I$ as required by condition Eq. (7):

$$\mathcal{L}_{\text{init}} = \sum_{x \in \mathcal{X}_I} \max(0, V(x) - \beta) \quad (10)$$

When $\mathcal{L}_{\text{init}}$ becomes 0, it means that $V(x) - \beta \leq 0$ for all data we sample from $\mathcal{X}_I$, then Eq. (7) is satisfied in the training data.

Descent Loss: We first introduce the descent loss without any perturbation proposed in [22], which enforces V decrease at least ϵ at each step before reaching the goal state as required by Eq. (3).

$$\mathcal{L}_{\text{dec}} = \sum_{x \in \mathcal{X} \setminus \mathcal{X}_G \wedge V(x) \leq \beta} \max(0, \epsilon - (V(x) - V(f(x, \pi(x)))) \quad (11)$$

When $\mathcal{L}_{\text{dec}}$ becomes 0, $\epsilon - (V(x) - V(f(x, \pi(x)))) \leq 0$ for all data we sampled outside $\mathcal{X}_G$ and filtered by the precondition $V(x) \leq \beta$. Then, Eq. (3) is satisfied in the training data.

4.2.1 Enforcing Robust Lyapunov Condition

The descent condition Eq. (11) ensures that the certificate value $V(x)$ decreases at least ϵ from the current state x to its next state $f(x, \pi(x))$. However, this ideal condition is not sufficient in practice,

where modeling errors, sensor noise, or environmental disturbances can perturb the next state. To address this, we introduce loss terms designed to enforce either empirical robustness or certified robustness. The key idea is to ensure that $V(x)$ remains ϵ larger than its value not only in the ideal next state but throughout a neighborhood around it, i.e. to satisfy Eq. (8) instead of Eq. (3). The loss terms described in the following each contribute to enforcing this property through different methods. Let y_{adv} be the point in the l_p -ball with radius δ around $f(x, \pi(x))$ that maximizes V :

$$y_{\text{adv}} = \arg \max_{y \in \mathcal{B}_{\delta,p}(f(x, \pi(x)))} V(y).$$

The robust descent condition Eq. (8) is then equivalent to requiring: $V(x) - V(y_{\text{adv}}) \geq \epsilon$. This equivalence follows from the fact that if $V(x)$ is at least ϵ greater than all values of $V(y)$ within the neighborhood, then it must also be greater than the maximum of these values by at least ϵ , vice versa. This formula enables us to design loss terms based either on an empirical estimate of $V(y_{\text{adv}})$, obtained via adversarial optimization, or on certified upper bounds derived from the Lipschitz continuity of V .

1. Adversarial Descent Loss. We first use adversarial attack strategies [9, 14, 21] to find $\hat{y}_{\text{adv}}$ to approximate y_{adv} , i.e., $V(\hat{y}_{\text{adv}}) \leq V(y_{\text{adv}})$. We design a loss term for the robust decrease condition:

$$\mathcal{L}_{\text{dec-adv}} = \sum_{x \in \mathcal{X} \setminus \mathcal{X}_G \wedge V(x) \leq \beta} \max(0, \epsilon - (V(x) - V(\hat{y}_{\text{adv}}))) \quad (12)$$

When this loss term comes to 0, it indicates that for every sampled state, the certificate value at the current state exceeds the approximated maximum value given by adversarial attack strategies within the perturbation neighborhood by at least ϵ , thus approximately satisfying the robust CLBF condition on the training data. The total training loss becomes:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{init}} \mathcal{L}_{\text{init}} + \lambda_{\text{dec-adv}} \mathcal{L}_{\text{dec-adv}}$$

where $\lambda_{\text{init}}, \lambda_{\text{dec-adv}} \in \mathbb{R}$ are the coefficients for the loss terms respectively.

There is no guarantee that adversarial attack strategies always find the worst-case perturbation. As a result, $V(\hat{y}_{\text{adv}})$ may underestimate the greatest value of V in $\mathcal{B}_{\delta,p}(f(x, \pi(x)))$. To overcome this limitation, we leverage the global Lipschitz constants of V . We introduce the following theorem connecting the Lipschitz constant with the robustness of the Lyapunov certificate:

Theorem 4.2. *Given a Lyapunov Barrier Certificate V defined following Definition 3.1 with parameters $(\alpha, \beta, \epsilon_r)$, if V is Lipschitz continuous under the l_p norm with global Lipschitz constant L_p , and any perturbation in the x_{t+1} is in $\mathcal{B}_{\delta,p}(f(x_t, \pi(x_t)))$, then the robust Lyapunov condition Eq. (8) holds with parameters $(\alpha, \beta, \epsilon_r - L_p \cdot \delta)$.*

Proof sketch. The Lipschitz continuity of V implies that for any perturbed next state y in the δ -ball around $f(x, \pi(x))$, $V(y)$ is at most $L_p \cdot \delta$ larger than $V(f(x, \pi(x)))$. Subtracting this from ϵ_r ensures that the robust decrease condition holds. The full proof is provided in Appendix B.4.

2. Lipschitz-bound Descent Loss. Instead of empirically searching for y_{adv} , we bound the maximum of $V(y)$ for $y \in \mathcal{B}_{\delta,p}(f(x, \pi(x)))$ using global Lipschitz bound L_p . This gives a modified descent loss:

$$\mathcal{L}_{\text{dec-neighbor}} = \sum_{x \in \mathcal{X} \setminus \mathcal{X}_G \wedge V(x) \leq \beta} \max(0, \epsilon - (V(x) - V(f(x, \pi(x))) - L_p \cdot \delta)). \quad (13)$$

Theorem 4.3. *If $\mathcal{L}_{\text{dec-neighbor}} = 0$, then Eq. (8) is satisfied on all sampled data.*

Proof sketch. When the loss is zero, we have $V(x) - V(f(x, \pi(x))) \geq \epsilon + L_p \cdot \delta$. By Theorem 4.2, this guarantees that $V(x) - V(y) \geq \epsilon$ for all $y \in \mathcal{B}_{\delta,p}(f(x, \pi(x)))$, thereby satisfying the robust condition. Full proof is in Appendix B.3.

We do not explicitly constrain the global Lipschitz constant of the certificate function V . Instead, we enforce a stronger descent condition by leveraging the connection between the Lipschitz continuity of a function and its local value differences. The total training loss becomes:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{init}} \mathcal{L}_{\text{init}} + \lambda_{\text{dec-neighbor}} \mathcal{L}_{\text{dec-neighbor}}$$

where $\lambda_{\text{init}}, \lambda_{\text{dec-neighbor}} \in \mathbb{R}$ are the coefficients for the loss terms, respectively.

3. Lipschitz Regularization Loss. While the loss above provides robustness for training data, it does not provide certified guarantees for all the states in $\mathcal{X}$. In safety-critical systems, however, we want robustness guarantees over the entire state space. To bridge this gap, we introduce the Lipschitz Regularization approach to achieve certifiable robustness on the full space. As given by Eq. (5), we use the product of spectral norms of each layer to upper bound the global Lipschitz constant in l_2 norm. To enforce an upper bound τ on the global Lipschitz constant, we use:

$$\mathcal{L}_{\text{lip-global}} = \max \left(0, \prod_{k=1}^{\ell} \|W_k\|_2 - \tau \right). \quad (14)$$

When this loss term reaches 0, we have $\prod_{k=1}^{\ell} \|W_k\|_2 \leq \tau$. According to Eq. (6), the Lipschitz constant under the l_p norm is bounded by $L_p = K_{p,2} \cdot \tau$, where $K_{p,2}$ is the norm conversion constant. From Theorem 4.2, if the Lyapunov Barrier certificate V satisfies the standard conditions in Definition 3.1 and has its Lipschitz constant bounded by L_p , then V also satisfies the robust Lyapunov Barrier Certificate definition in Definition 4.2 with $\epsilon = \epsilon_r - \delta \cdot L_p$. We aim to minimize the Lipschitz constant L_p because the robust condition $\epsilon > 0$ implies the constraint $\delta < \epsilon_r / L_p$. This means that a smaller L_p allows for a larger admissible perturbation radius δ , thus improving the certified robustness. To determine the smallest achievable Lipschitz bound τ that still satisfies the standard certificate conditions Eq. (2)–(4), we perform a binary search of τ between a lower bound of 0 and an upper bound given by the Lipschitz constant of a vanilla model trained to satisfy conditions the standard conditions. Since this vanilla model is already known to satisfy the standard conditions, the upper bound is guaranteed to be feasible. Since our algorithm ensures the Lipschitz bound τ is met and the certificate conditions can be proved to be satisfied using the verifier when it finishes, we can guarantee that the resulting certificate is robust under perturbations of radius δ .

In the training process, we use the original training loss $\mathcal{L}_{\text{init}}$ and $\mathcal{L}_{\text{dec}}$ to satisfy the condition in Eq. (2)–(4), and add the loss term $\mathcal{L}_{\text{lip-global}}$ to upper bound the global Lipschitz constant. The total training loss becomes:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{init}} \mathcal{L}_{\text{init}} + \lambda_{\text{dec}} \mathcal{L}_{\text{dec}} + \lambda_{\text{lip-global}} \mathcal{L}_{\text{lip-global}}$$

where $\lambda_{\text{init}}, \lambda_{\text{dec}}, \lambda_{\text{lip-global}} \in \mathbb{R}$ are the coefficients for the loss terms, respectively. Since we have the loss to enforce Eq. (2)–(4) are satisfied, we do not need to explicitly enforce the robust decrease condition Eq. (8). Instead, according to Theorem 4.2, by bounding the Lipschitz constant and satisfying the nominal decrease condition, we ensure that the robust condition is also satisfied.

5 Experiments

In this section, we first describe the experimental setup. We then present results on both certified and empirical robustness, followed by an ablation study to analyze the impact of key hyperparameters. Each experiment is run multiple times, and detailed statistics are provided in Appendix E.1.

5.1 Experiment Setup

We validate our method on two reinforcement learning environments: the *2D Docking* task, which captures important safety-critical behaviors relevant to autonomous systems, and the *Inverted Pendulum*, a popular benchmark in control theory. A detailed case study is provided in Appendix C. Following prior work in [22, 44], We design the policy network for the 2D docking task with two hidden layers of 20 neurons each, and the policy for the inverted pendulum environment with two hidden layers of 128 neurons each, both using ReLU activations. The certificate network consists of three hidden layers with 64, 32, and 16 neurons respectively, also using ReLU activations. All experiments were conducted on a server equipped with 12 cores of Intel Xeon Silver 4214R CPUs, 187 GB of RAM, and running Ubuntu 22.04.

We set $\beta = 1$ in both environments, and mask the certificate value such that $V(x) = -10$ for goal states $x \in \mathcal{X}_G$ and $V(x) = 1.2$ for unsafe states $x \in \mathcal{X}_U$. The descent margin ϵ is selected by training the non-robust Lyapunov Barrier Certificate (satisfying only Eq. (2)–(4)) with the largest possible margin until the certificate cannot be learned. We then keep this value fixed for robust training: $\epsilon = 10^{-2}$ for 2D Docking and 5×10^{-3} for Inverted Pendulum. For each method, we binary search for the largest Lipschitz bound τ and neighborhood radius δ that allow training to converge. In

Method	Certified Bound
Vanilla	0.0015
Lip-3	0.0084
Lip-Neighbor	0.0033
PGD	0.0078

(a) Inverted Pendulum environment

Method	Certified Bound
Vanilla	0.0094
Lip-1	0.0172
Lip-Neighbor	0.0156
PGD	0.0131

(b) 2D Docking environment

Table 1: Certified perturbation bounds for different training methods. **Method:** Vanilla denotes a baseline model trained using the standard method without robustness. Lip- n corresponds to training with Lipschitz bound $\tau = n$ as in Eq. (14). Lip-Neighbor uses a neighborhood margin of $\delta = 0.01$ (2D docking) or $\delta = 10^{-3}$ (Inverted pendulum) as in Eq. (13). PGD is trained with adversarial examples in the perturbation range of $\delta = 0.01$ (2D docking) or $\delta = 5 \times 10^{-3}$.

the training process, we set $\lambda_{\text{init}} = 1$, $\lambda_{\text{lip-global}} = 1$, while all other loss weights (i.e., λ_{dec} , $\lambda_{\text{dec-adv}}$, $\lambda_{\text{dec-neighbor}}$) are set to 10, depending on the specific training objective being used. We conducted our experiment on l_{∞} norm. In the adversarial training method, we use Projected Gradient Descent (PGD) [21] to approximate the value of y_{adv} , the point in the neighborhood where V is maximized.

In the CEGIS loop, we train until a loss of 0 is achieved and then use the verification step to find counterexamples. We repeat this until there are no more counterexamples or a timeout (12 hours) is reached. We use the Marabou verifier [41] in the verification step. When the system dynamics involve operations that cannot be precisely analyzed by existing DNN verifiers (e.g., trigonometric functions), we encode piecewise-linear bounds that over-approximate the dynamics. The bounds are constructed with OVERT [31], with the number of piecewise-linear segments set to 3 and an absolute approximation error tolerance set to $\epsilon = 10^{-5}$.

5.2 Certified robustness

Certifiable robustness is evaluated using the Marabou DNN verifier [41] with a binary search procedure to find the largest δ for which Eq. (7)–(9) hold with $\epsilon = 10^{-6}$. We repeat the binary search procedure until the interval length is below 10^{-4} , and we then report the lower bound as the certified perturbation. Tables 1a and 1b show the certified bounds for the controller in Inverted Pendulum and 2D Docking environments, representing the maximum perturbation under which the controller provably complete in the robust RWA task. Compared to the baseline (Vanilla), our robust training methods improve certified robustness. In the Inverted Pendulum environment, Lip-3 improves the bound by 460% (from 0.0015 to 0.0084), while in 2D Docking, Lip-1 increases it by 83%. Since perturbations are measured under the l_{∞} norm, certified volume grows exponentially with state dimension—Lip-3 increases the volume over 30 $\times$ in 2D, and Lip-1 achieves over 10 $\times$ in 4D.

5.3 Empirical robustness

In addition to certifiable bounds, we also evaluate empirical robustness against both adversarial and random perturbations. After training, we simulate the controllers under adversarial perturbations and random perturbations. For adversarial perturbations, we apply attacks based on maximizing the certificate value in each step; for random perturbations, we apply bounded noise to the state computed by the environment at each time step. Tables 2a and 2b show the empirical success rates across perturbation ranges in both environments. A trajectory is considered successful if it reaches the goal region within 200 steps without entering the unsafe region. We report the success rate over 10,000 randomly sampled states that are in initial region but outside of the goal region. For each method, we train two models and report the average success rate.

In both environments, all proposed methods significantly outperform the Vanilla baseline. In the Inverted Pendulum environment, under a perturbation of 0.01, Lip-3 and Lip-Neighbor achieve success rates of 70% and 69%, nearly doubling the baseline’s 39%, while PGD achieves the highest success at 89%. In the 2D Docking environment, both Lipschitz-based methods (Lip-1 and Lip-Neighbor) maintain 100% success up to 0.03 perturbation—2.4 $\times$ better than Vanilla (41%)—and retain strong performance at 0.05 (83% and 86%). These comparisons clearly indicate that our methods not only exceed Vanilla performance within its certifiable perturbation radius, but also generalize substantially better under stronger perturbations.

Method	0.005 Adv	0.008 Adv	0.01 Adv	0.05 Rand
Vanilla	72%	41%	39%	100%
Lip-3	100%	100%	70%	100%
Lip-Neighbor	78%	68%	69%	100%
PGD	100%	100%	89%	100%

(a) Inverted Pendulum environment

Method	0.01 Adv	0.03 Adv	0.05 Adv	0.05 Rand
Vanilla	100%	41%	34%	100%
Lip-1	100%	100%	83%	100%
Lip-Neighbor	100%	100%	86%	100%
PGD	100%	51%	17%	100%

(b) 2D Docking environment

Table 2: Success rates (%) under various adversarial and random perturbations in both environments. " x Adv" indicates adversarial perturbations with ℓ_∞ norm bound x applied using PGD attacks. " x Rand" denotes uniformly random perturbations within the range applied at each time step.

PGD-trained models achieve high success rates within the perturbation radius they were trained on, but their performance significantly drops under larger perturbations. This highlights a well-known limitation of adversarial training that it does not generalize well to unseen, larger perturbations. On the other hand, methods that incorporate Lipschitz constants show improved robustness across a wide range of perturbations. This is because these methods either implicitly (Lipschitz-Neighbor, bounding the maximum value using Lipschitz constant) or explicitly (Lip-n, upper-bounding the Lipschitz constant by n) enforce a smaller Lipschitz bound, encouraging smoother behavior in the neural network. As a result, small perturbations in the state lead to only minor changes in the certificate value. When the certificate value remains close to the value before perturbed, the decrease condition is more likely to hold, thereby improving the system’s ability to remain safe and reach the goal state.

5.4 Ablation Study

We conduct an ablation study on the 2D docking setting to understand the influence of key hyperparameters on training performance and certified robustness. Full numerical results are in Appendix E.2.

Effect of Robust Descent Margin ϵ . We vary $\epsilon \in \{0.001, 0.005, 0.01\}$ in the Lipschitz Regularization method. We find that smaller margins lead to lower certified robustness with roughly the same training time, validating our approach of choosing the largest feasible ϵ from the vanilla model.

Effect of Lipschitz Upper Bound τ . We evaluate several values of τ in the Lipschitz Regularization method. Lower τ generally leads to improved certified robustness, consistent with the theoretical guarantee that a smaller Lipschitz constant allow certification under larger perturbations. However, setting τ too small makes the learning process difficult to converge.

Effect of Loss Weight $\lambda_{\text{lip-global}}$. To study the impact of the Lipschitz loss weight in the Lipschitz Regularization method, we fix $\lambda_{\text{init}} = 1$, $\lambda_{\text{dec}} = 10$ and vary $\lambda_{\text{lip}} \in \{0.5, 1, 5, 10, 20\}$. We observe that training time first decreases and then increases, and certified robustness first decreases and then increases. This suggests the balance between regularization strength and optimization dynamics.

Effect of Counterexample Weighting w . During CEGIS training, we test different loss weight ratios w between original and counterexample where $w \in \{100, 500, 1000\}$. While giving higher weight to counterexamples helps training converge fast, setting the ratio too high reduces overall robustness, probably due to overfitting to the counterexamples.

6 Conclusion and Limitations

In this paper, we studied the problem of synthesizing robust neural Lyapunov barrier certificate, where the validity of the certificate holds even under norm-bounded time-varying perturbations in the system dynamics. We established the connection between the robustness of the certificate and Lipschitz continuity, and proposed practical training methods, including adversarial training and Lipschitz-based methods, to enforce robustness for synthesizing robust certificates. We showed that our approach improved both certifiable and empirical robustness in two environments: Inverted Pendulum, a popular benchmark for control, and 2D Docking, a safety-critical task relevant to autonomous systems. A limitation of our work is its focus on robustness to perturbations in the states only. In practice, robustness may be required against a broader range of uncertainty, including perturbations in the system dynamics, observations, or deviations in the controller’s output. Addressing these sources of uncertainty is an important direction for future work, and could enhance the framework’s effectiveness in real-world settings, where sim-to-real gaps often arise from such unmodeled variations.

References

- [1] Alessandro Abate, Daniele Ahmed, Alec Edwards, Mirco Giacobbe, and Andrea Peruffo. Fossil: a software tool for the formal synthesis of lyapunov functions and barrier certificates using neural networks. In *Proceedings of the 24th international conference on hybrid systems: computation and control*, pages 1–11, 2021.
- [2] Alessandro Abate, Daniele Ahmed, Mirco Giacobbe, and Andrea Peruffo. Formal synthesis of lyapunov neural networks. *IEEE Control Systems Letters*, 5(3):773–778, 2020.
- [3] Aaron D Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. Control barrier functions: Theory and applications. In *2019 18th European control conference (ECC)*, pages 3420–3431. Ieee, 2019.
- [4] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.
- [5] Manuel Cardona and Fernando E. Serrano. *Robust Control Lyapunov Functions*, pages 127–140. Springer Nature Switzerland, Cham, 2025.
- [6] Ya-Chien Chang, Nima Roohi, and Sicun Gao. Neural lyapunov control. *Advances in neural information processing systems*, 32, 2019.
- [7] Moustapha Cisse, Piotr Bojanowski, Edouard Grave, Yann Dauphin, and Nicolas Usunier. Parseval networks: Improving robustness to adversarial examples. In *International conference on machine learning*, pages 854–863. PMLR, 2017.
- [8] Pierre Clavier, Stéphanie Allasonière, and Erwan Le Pennec. Robust reinforcement learning with distributional risk-averse formulation. *arXiv preprint arXiv:2206.06841*, 2022.
- [9] Francesco Croce and Matthias Hein. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. In *International conference on machine learning*, pages 2206–2216. PMLR, 2020.
- [10] Charles Dawson, Zengyi Qin, Sicun Gao, and Chuchu Fan. Safe nonlinear control using robust neural lyapunov-barrier functions. In *Conference on Robot Learning*, pages 1724–1735. PMLR, 2022.
- [11] RA Freeman and PV Kokotovic. Robust control lyapunov functions: The measurement feedback case. In *Proceedings of 1994 33rd IEEE Conference on Decision and Control*, volume 4, pages 3533–3538. IEEE, 1994.
- [12] Kunal Garg and Dimitra Panagou. Robust control barrier and control lyapunov functions with fixed-time convergence guarantees. In *2021 American Control Conference (ACC)*, pages 2292–2297. IEEE, 2021.
- [13] Peter Giesl and Sigurdur Hafstein. Review on computational methods for lyapunov functions. *Discrete and Continuous Dynamical Systems-B*, 20(8):2291–2331, 2015.
- [14] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.
- [15] Wassim M Haddad and VijaySekhar Chellaboina. *Nonlinear dynamical systems and control: a Lyapunov-based approach*. Princeton university press, 2008.
- [16] Dmitry Kalashnikov, Alex Irpan, Peter Pastor, Julian Ibarz, Alexander Herzog, Eric Jang, Deirdre Quillen, Ethan Holly, Mrinal Kalakrishnan, Vincent Vanhoucke, et al. Scalable deep reinforcement learning for vision-based robotic manipulation. In *Conference on robot learning*, pages 651–673. PMLR, 2018.
- [17] Mathias Lechner, Đorđe Žikelić, Krishnendu Chatterjee, and Thomas A Henzinger. Stability verification in stochastic control systems via neural network supermartingales. In *Proceedings of the aaai conference on artificial intelligence*, volume 36, pages 7326–7336, 2022.

- [18] Klas Leino, Zifan Wang, and Matt Fredrikson. Globally-robust neural networks. In *International Conference on Machine Learning*, pages 6212–6222. PMLR, 2021.
- [19] Shiao Hong Lim, Huan Xu, and Shie Mannor. Reinforcement learning in robust markov decision processes. *Advances in Neural Information Processing Systems*, 26, 2013.
- [20] Simin Liu, John Dolan, and Changliu Liu. Safe control under input saturation with neural control barrier functions. In *6th Annual Conference on Robot Learning*, volume 1, pages 2–2, 2022.
- [21] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. *arXiv preprint arXiv:1706.06083*, 2017.
- [22] Udayan Mandal, Guy Amir, Haoze Wu, Ieva Daukantas, Fletcher Lee Newell, Umberto J Ravaioli, Baoluo Meng, Michael Durling, Milan Ganai, Tobey Shim, et al. Formally verifying deep reinforcement learning controllers with lyapunov barrier certificates. In *# PLACE-HOLDER_PARENT_METADATA_VALUE#*, pages 95–106. TU Wien Academic Press, 2024.
- [23] Ajay Mandlekar, Yuke Zhu, Animesh Garg, Li Fei-Fei, and Silvio Savarese. Adversarially robust policy learning: Active construction of physically-plausible perturbations. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3932–3939. IEEE, 2017.
- [24] Frederik Baymler Mathiesen, Simeon C Calvert, and Luca Laurenti. Safety certification for stochastic systems via neural barrier functions. *IEEE Control Systems Letters*, 7:973–978, 2022.
- [25] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- [26] Jun Morimoto and Kenji Doya. Robust reinforcement learning. *Neural computation*, 17(2):335–359, 2005.
- [27] Xinlei Pan, Daniel Seita, Yang Gao, and John Canny. Risk averse robust adversarial reinforcement learning. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 8522–8528. IEEE, 2019.
- [28] Lerrel Pinto, James Davidson, Rahul Sukthankar, and Abhinav Gupta. Robust adversarial reinforcement learning. In *International conference on machine learning*, pages 2817–2826. PMLR, 2017.
- [29] Umberto J Ravaioli, James Cunningham, John McCarroll, Vardaan Gangal, Kyle Dunlap, and Kerianne L Hobbs. Safe reinforcement learning benchmark environments for aerospace control systems. In *2022 IEEE Aerospace Conference (AERO)*, pages 1–20. IEEE, 2022.
- [30] Ahmad EL Sallab, Mohammed Abdou, Etienne Perot, and Senthil Yogamani. Deep reinforcement learning framework for autonomous driving. *arXiv preprint arXiv:1704.02532*, 2017.
- [31] Chelsea Sidrane, Amir Maleki, Ahmed Irfan, and Mykel J Kochenderfer. Overt: An algorithm for safety verification of neural network control policies for nonlinear systems. *Journal of Machine Learning Research*, 23(117):1–45, 2022.
- [32] Armando Solar-Lezama. *Program synthesis by sketching*. University of California, Berkeley, 2008.
- [33] Samuel Stanton, Rasool Fakoor, Jonas Mueller, Andrew Gordon Wilson, and Alex Smola. Robust reinforcement learning for shifting dynamics during deployment. 2021.
- [34] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. *arXiv preprint arXiv:1312.6199*, 2013.

- [35] Dursun Tasci. On the relations between l_p norms for matrices. *Communications de la Faculté des Sciences de l'Université d'Ankara. Séries A1: Mathematics and Statistics*, 01 1994.
- [36] Chen Tessler, Yonathan Efroni, and Shie Mannor. Action robust reinforcement learning and applications in continuous control. In *International Conference on Machine Learning*, pages 6215–6224. PMLR, 2019.
- [37] Yusuke Tsuzuku, Issei Sato, and Masashi Sugiyama. Lipschitz-margin training: Scalable certification of perturbation invariance for deep neural networks. *Advances in neural information processing systems*, 31, 2018.
- [38] Eugene Vinitzky, Yuqing Du, Kanaad Parvate, Kathy Jang, Pieter Abbeel, and Alexandre Bayen. Robust reinforcement learning using adversarial populations. *arXiv preprint arXiv:2008.01825*, 2020.
- [39] Kaixin Wang, Uri Gadot, Navdeep Kumar, Kfir Yehuda Levy, and Shie Mannor. Robust reinforcement learning via adversarial kernel approximation. In *Sixteenth European Workshop on Reinforcement Learning*, 2023.
- [40] Wolfram Wiesemann, Daniel Kuhn, and Berç Rustem. Robust markov decision processes. *Mathematics of Operations Research*, 38(1):153–183, 2013.
- [41] Haoze Wu, Omri Isac, Aleksandar Zeljić, Teruhiro Tagomori, Matthew Daggitt, Wen Kokke, Idan Refaeli, Guy Amir, Kyle Julian, Shahaf Bassan, et al. Marabou 2.0: a versatile formal analyzer of neural networks. In *International Conference on Computer Aided Verification*, pages 249–264. Springer, 2024.
- [42] Huan Zhang, Hongge Chen, Duane Boning, and Cho-Jui Hsieh. Robust reinforcement learning on state observations with learned optimal adversary. *arXiv preprint arXiv:2101.08452*, 2021.
- [43] Huan Zhang, Hongge Chen, Chaowei Xiao, Bo Li, Mingyan Liu, Duane Boning, and Cho-Jui Hsieh. Robust deep reinforcement learning against adversarial perturbations on state observations. *Advances in Neural Information Processing Systems*, 33:21024–21037, 2020.
- [44] Đorđe Žikelić, Mathias Lechner, Thomas A Henzinger, and Krishnendu Chatterjee. Learning control policies for stochastic systems with reach-avoid guarantees. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 11926–11935, 2023.

A Additional Related Work

A.1 Lipschitz Continuity

Lipschitz continuity has been widely explored in the machine learning literature as a tool to improve model robustness, particularly in classification tasks. Many works use Lipschitz constant to partially explain the sensitivity of neural networks [34] or enforce global Lipschitz bound during training to improve robustness [7, 18, 37]. However, these approaches are not directly applicable to control systems or Lyapunov-based verification, where the goal is not classification accuracy but certified state-space behavior over time. Our use of Lipschitz continuity focuses on certifying robustness under state perturbations in dynamic systems.

A.2 Safe Reinforcement Learning

Robust RL methods aim to improve policy performance under perturbations. Some approaches adopt the robust Markov Decision Process formulation [19, 40], defining uncertainty sets around nominal dynamics [8, 39] and solve robust Bellman updates. A more widely adopted perspective models robustness as a minimax game against adversaries, first introduced in [26]. Adversarial training frameworks build on this idea, employing single [28, 36], population [38], or risk-aware adversaries [27] to simulate worst-case perturbations on policies and improve robustness. Other lines of work consider adversarial perturbations in the observation space [33, 42, 43]. While effective for improving empirical performance, these methods typically do not provide formal, trajectory-level robustness guarantees. In contrast, our approach certifies safety and stability by synthesizing neural Lyapunov-barrier functions that provably satisfy robustness under norm-bounded perturbations.

B Proof of Theorems

B.1 Norm Inequality Derivation

To generalize to other l_p norms, we use the following norm inequality:

$$\|x\|_p \leq n^{(\frac{1}{p}-\frac{1}{q})} \|x\|_q.$$

Applying this to both the input and output spaces, we obtain a Lipschitz constant for the function f in l_q -input and l_p -output norms:

$$\|f(y) - f(x)\|_p \leq m^{(\frac{1}{p}-\frac{1}{2})} \|f(y) - f(x)\|_2 \quad (15)$$

$$\leq m^{(\frac{1}{p}-\frac{1}{2})} L_2 \|y - x\|_2 \quad (16)$$

$$\leq m^{(\frac{1}{p}-\frac{1}{2})} L_2 \cdot n^{(\frac{1}{2}-\frac{1}{q})} \|y - x\|_q, \quad (17)$$

where n and m are the input and output dimensions of f respectively, and L_2 is the global Lipschitz constant of f under the l_2 norm. Thus, the overall bound becomes:

$$\|f(y) - f(x)\|_p \leq n^{(\frac{1}{2}-\frac{1}{q})} m^{(\frac{1}{p}-\frac{1}{2})} L_2 \|y - x\|_q. \quad (18)$$

B.2 Proof of Theorem B.1

Theorem B.1 (Robust RWA Guarantee). *Let $\{x_t\}_{t=0}^\infty$ be a trajectory generated by policy π and transition function f , and the perturbation at each state is less than δ under l_p norm, i.e., $x_{t+1} \in \mathcal{B}_{\delta,p}(f(x_t, \pi(x_t)))$. If there exists a (δ, p) -Robust Lyapunov Barrier Certificate $V : \mathcal{X}_s \rightarrow \mathbb{R}_{\geq c}$ where $\mathcal{X}_s = \{x_\tau \mid \forall t \in [0, \tau - 1], x_t \notin \mathcal{X}_G\} \subset \mathcal{X}$ with parameters $(\alpha, \beta, \epsilon)$, satisfies condition Eq. (7)–(9), then the trajectory: (i) will reach the goal region $\mathcal{X}_G$ in finite time and (ii) will never enter the unsafe region $\mathcal{X}_U$, thereby completing the robust RWA task under state-wise perturbations.*

Proof. To prove (i), assume by contradiction that the trajectory never reaches $\mathcal{X}_G$. Then $x_t \in \mathcal{X}_s$ for all $t \in \mathbb{N}$, and by Eq. (8) we have:

$$V(x_t) - V(x_{t+1}) \geq \epsilon > 0.$$

This implies $V(x_t)$ strictly decreases by at least ϵ_r at every step. Since $V(x) \geq c$ for all $x \in \mathcal{X}_s$, and $V(x_0) \leq \beta$, the number of steps before reaching a contradiction is at most

$$T \leq \frac{\beta - c}{\epsilon}.$$

Therefore, the trajectory must reach $\mathcal{X}_G$ in finite time.

To prove (ii), let τ be the first time $x_\tau \in \mathcal{X}_G$. For all $t \leq \tau$, we have $x_t \in \mathcal{X}_s$, and from Eq. (8):

$$V(x_t) < V(x_{t-1}) < \dots < V(x_0) \leq \beta.$$

Since $\beta < \alpha$, it follows that $V(x_t) < \alpha$, so $x_t \notin \mathcal{X}_U$. Thus, the trajectory avoids the unsafe region at all times. $\square$

B.3 Proof of Theorem B.2

Theorem B.2. $\mathcal{L}_{\text{dec-neighbor}}$ is defined by:

$$\mathcal{L}_{\text{dec-neighbor}} = \sum_{x \in \mathcal{X} \setminus \mathcal{X}_G \wedge V(x) \leq \beta} \max(0, \epsilon - (V(x) - V(f(x, \pi(x))) - L_p \cdot \delta)).$$

If $\mathcal{L}_{\text{dec-neighbor}} = 0$, then Eq. (8) is satisfied on all sampled data.

Proof. Let $x_{\text{next}} = f(x, \pi(x))$. By the definition of the Lipschitz constant of V under the l_p -norm, we have

$$\forall y \in \mathcal{B}_{\delta,p}(x_{\text{next}}), \|V(y) - V(x_{\text{next}})\|_p \leq L_p \cdot \|y - x_{\text{next}}\|_p \leq L_p \cdot \delta.$$

Since $V(y), V(x_{\text{next}}) \in \mathbb{R}$, we have: $V(y) - V(x_{\text{next}}) \leq \|V(y) - V(x_{\text{next}})\|_p$. Thus,

$$V(y) \leq V(x_{\text{next}}) + L_p \cdot \delta \tag{19}$$

$$\begin{aligned} \mathcal{L}_{\text{dec-neighbor}} = 0 &\iff \forall x \in \mathcal{X} \setminus \mathcal{X}_G \wedge V(x) \leq \beta, \max(0, \epsilon - (V(x) - L_p \cdot \delta - V(x_{\text{next}}))) = 0 \\ &\iff \forall x \in \{x \in \mathcal{X} \setminus \mathcal{X}_G \mid V(x) \leq \beta\}, V(x) - L_p \cdot \delta - V(x_{\text{next}}) \geq \epsilon \end{aligned}$$

Applying Eq. (19) we get:

$$\forall x \in \{x \in \mathcal{X} \setminus \mathcal{X}_G \mid V(x) \leq \beta\}, \forall y \in \mathcal{B}_{\delta,p}(x_{\text{next}}), V(x) - V(y) \geq \epsilon,$$

which is the same as Eq. (8). $\square$

B.4 Proof of Theorem B.3

Theorem B.3. Given a Lyapunov Barrier Certificate V defined following Definition 3.1 with parameters $(\alpha, \beta, \epsilon_r)$, if V is Lipschitz continuous under the l_p norm with global Lipschitz constant L_p , and any perturbation in the x_{t+1} is in $\mathcal{B}_{\delta,p}(f(x_t, \pi(x_t)))$, then the robust Lyapunov condition Eq. (8) holds with parameters $(\alpha, \beta, \epsilon_r - L_p \cdot \delta)$.

Proof. Let $x_{\text{next}} = f(x, \pi(x))$ denote the unperturbed next state. Suppose the perturbed next state is $y \in \mathcal{B}_{\delta,p}(x_{\text{next}})$, i.e.,

$$\|y - x_{\text{next}}\|_p \leq \delta.$$

By Lipschitz continuity of V , we have:

$$\|V(y) - V(x_{\text{next}})\|_p \leq L_p \cdot \|y - x_{\text{next}}\|_p \leq L_p \cdot \delta.$$

From Eq. (3) in Definition 3.1:

$$V(x_{\text{next}}) \leq V(x) - \epsilon_r,$$

we get:

$$V(y) \leq V(x_{\text{next}}) + L_p \cdot \delta \leq V(x) - \epsilon_r + L_p \cdot \delta.$$

Rewriting this, we get:

$$V(y) \leq V(x) - (\epsilon_r - L_p \cdot \delta),$$

which proves that the robust decrease condition Eq. (8) holds under perturbations. $\square$

C Case Study

Inverted Pendulum The Inverted Pendulum environment is based on OpenAI Gym [4]. The task is to drive the pendulum to the upright position with near-zero velocity and avoid falling down during the process. Specifically, the state of the system is $\mathbf{x} = [\theta, \dot{\theta}]$, where θ is the angle between the pendulum and the upright vertical, and $\dot{\theta}$ is the angular velocity. The control input $u \in [-1, 1]$ is the applied torque determined by the policy $\pi(\mathbf{x})$. The dynamic system is:

$$\begin{aligned}\dot{\theta}' &= (1 - b)\dot{\theta} + \left(\frac{1.5 \cdot g \cdot \sin \theta}{2 \cdot l} + \frac{3}{ml^2} \cdot 2u \right) \cdot T \\ \theta' &= \theta + \dot{\theta}' \cdot T\end{aligned}$$

with $g = 10 \text{ m/s}^2$, $m = 0.15 \text{ kg}$, $l = 0.5 \text{ m}$, $b = 0.1$, $T = 0.05 \text{ s}$. The definitions of state sets follow the previous work [44] with $\mathcal{X} = [-0.7, 0.7]^2$, $\mathcal{X}_I = [-0.3, 0.3]^2$, $\mathcal{X}_G = [-0.2, 0.2]^2$, $\mathcal{X}_U = [-0.7, -0.6] \times [-0.7, 0] \cup [0.6, 0.7] \times [0, 0.7]$.

2D Docking The 2D Docking task is taken from [29]. Specifically, the task is to drive the deputy spacecraft, controlled with thrusters that provide forces in the x and y directions, to safely reach the goal state that is in close proximity to the chief spacecraft, while avoiding entering unsafe states. The state is $\mathbf{x} = [x, y, v_x, v_y]$, where (x, y) is position and (v_x, v_y) is velocity. The control input $\mathbf{u} = [F_x, F_y] \in [-1, 1]^2$ are the thrust forces applied along the x and y directions respectively, which is given by policy $\pi(\mathbf{x})$. The dynamic system given by transition function is:

$$\begin{aligned}x' &= \left(\frac{2v_y}{n} + 4x + \frac{F_x}{mn^2} \right) + \left(\frac{2F_y}{mn} \right) \\ &\quad + \left(-\frac{F_x}{mn^2} - \frac{2v_y}{n} - 3x \right) \cos(nT) + \left(\frac{-2F_y}{mn^2} + \frac{v_x}{n} \right) \sin(nT)\end{aligned}\quad (20)$$

$$\begin{aligned}y' &= \left(-\frac{2v_x}{n} + y + \frac{4F_y}{mn^2} \right) + \left(\frac{-2F_x}{mn} - 3v_y - 6nx \right) T - \frac{3F_y}{2m} t^2 \\ &\quad + \left(-\frac{4F_y}{mn^2} + \frac{2v_x}{n} \right) \cos(nT) + \left(\frac{2F_x}{mn^2} + \frac{4v_y}{n} + 6x \right) \sin(nT)\end{aligned}\quad (21)$$

$$v'_x = \left(\frac{2F_x}{mn} \right) + \left(\frac{-2F_y}{mn} + x \right) \cos(nT) + \left(\frac{F_x}{mn} + 2v_y + 3nx \right) \sin(nT)\quad (22)$$

$$\begin{aligned}v'_y &= \left(\frac{-2F_x}{mn} - 3v_y - 6nx \right) + \left(-\frac{3F_y}{m} \right) T \\ &\quad + \left(\frac{2F_x}{mn} + 4v_y + 6nx \right) \cos(nT) + \left(\frac{4F_y}{mn} - 2v_x \right) \sin(nT)\end{aligned}\quad (23)$$

with $m = 12 \text{ kg}$, $n = 0.001027 \text{ rad/s}$, $T = 1 \text{ s}$, and F_x, F_y between -1 and +1 Newtons. $\mathbf{x} = [x, y, v_x, v_y]$ is the current state and $\mathbf{x}' = [x', y', v'_x, v'_y]$ is the next state given by transition function f . Following [22], the state sets are defined as $\mathcal{X} = \mathbb{R}^4$, $\mathcal{X}_{safe} = [-2, 2]^2 \times [-0.5, 0.5]^2$, $\mathcal{X}_U = \mathcal{X} \setminus \mathcal{X}_{safe}$, $\mathcal{X}_G = [-0.35, 0.35]^2 \times \mathbb{R}^2$, $\mathcal{X}_I = [-1, 1]^2 \times [0, 0]^2$.

D Training Algorithm

Concretely, we sample data from the state space and use loss functions to penalize the violation of the condition in Eq. (7)–(9). Note that $\mathcal{X}_U \cap \mathcal{X}_I = \emptyset$ and $\forall x \in \mathcal{X}_U, V(x) > \beta$, so $x \in \mathcal{X}_U$ only influences Eq. (9). For simplicity of training, we do not add this constraint during training. Instead, we output a filtered certificate [22] on the certificate we get from the training, in which all the states $x \in \mathcal{X}_U$ are assigned to e where $e > \alpha$. The filtered certificate V then satisfies Eq. (9) for all $x \in \mathcal{X}_U$, so we do not need sample from $\mathcal{X}_U$. There is no constraint for $x \in \mathcal{X}_G$, so we also do not need sample data from $\mathcal{X}_G$. We only need to consider Eq. (7) and Eq. (8).

We first train an initial controller policy π via a reinforcement learning algorithm. Next, we perform several training epochs in which only the parameters of the certificate function V are updated to

initialize it, before jointly training both the controller and the certificate. Then in the CEGIS loop, every iteration we first train the controller policy π and certificate V together to minimize the total loss $\mathcal{L}_{\text{total}}$, which enforces the controller and certificate to satisfy Eq. (7) and Eq. (8). We describe the design of loss functions in Section 4.2. Once the loss reaches 0, it means that all the training data satisfy these conditions, then we use a neural network verifier to search for counterexamples that violate these conditions. If such counterexamples are not found, we get the verified controller and certificate. Otherwise, we augment the training dataset with samples from their neighborhoods and retrain the controller and certificate. This local resampling is motivated by the Lipschitz continuity of both the controller and the certificate: If a violation occurs at one state, nearby states are also likely to violate the conditions.

Algorithm 1 CEGIS Loop for Robust Lyapunov Barrier Certificate

```

1: Input:
2:   Environment dynamics  $f$ 
3:   State sets  $\mathcal{X}_I \leftarrow$  initial states,  $\mathcal{X}_G \leftarrow$  goal states,  $\mathcal{X}_U \leftarrow$  unsafe states
4:    $\mathcal{D}_{\text{orig}} \leftarrow$  training data sampled from  $\mathcal{X} \setminus \{\mathcal{X}_G \cup \mathcal{X}_U\}$ 
5:    $\beta \leftarrow$  initial certificate threshold
6:    $\epsilon \leftarrow$  descent margin
7:    $w \leftarrow$  loss weight for counterexample data
8:    $\lambda_* = \{\lambda_{\text{init}}, \lambda_{\text{dec}}, \lambda_{\text{dec-adv}}, \lambda_{\text{dec-neighbor}}, \lambda_{\text{lip-global}}\} \leftarrow$  training loss weights
9:    $\tau \leftarrow$  upper bound of Lipschitz constant, for Lipschitz Regularization method
10:  Verifier  $\leftarrow$  neural network verifier
11: Initialize:
12:   $\pi \leftarrow$  trained by RL algorithm
13:   $V \leftarrow$  trained by minimizing  $\mathcal{L}_{\text{total}}(\beta, \epsilon, \tau, \lambda_*)$  on dataset  $\mathcal{D}_{\text{orig}}$ 
14:   $\mathcal{D}_{\text{ce}} \leftarrow \emptyset$ 
15: repeat
16:   Training:
17:    $\mathcal{L}_{\text{orig}} \leftarrow \mathcal{L}_{\text{total}}(\beta, \epsilon, \tau, \lambda_*)$  calculated on dataset  $\mathcal{D}_{\text{orig}}$ 
18:    $\mathcal{L}_{\text{ce}} \leftarrow \mathcal{L}_{\text{total}}(\beta, \epsilon, \tau, \lambda_*)$  calculated on dataset  $\mathcal{D}_{\text{ce}}$  if  $\mathcal{D}_{\text{ce}} \neq \emptyset$  else 0
19:    $\mathcal{L}_{\text{final}} \leftarrow \mathcal{L}_{\text{orig}} + w \cdot \mathcal{L}_{\text{ce}}$ 
20:    $(V, \pi) \leftarrow$  trained by minimizing  $\mathcal{L}_{\text{final}}$ 
21:   Verification:
22:   counterexamples  $\leftarrow$  Verifier( $V_\theta, \pi_\phi$ )
23:   if counterexamples  $\neq \emptyset$  then
24:     Refinement:
25:      $\mathcal{D}_{\text{add}} \leftarrow$  uniformly sample  $m$  data points in  $l_p$  ball around each counterexample
26:      $\mathcal{D}_{\text{ce}} \leftarrow \mathcal{D}_{\text{ce}} \cup \mathcal{D}_{\text{add}}$ 
27:   else
28:     break
29: until max iterations reached  $\vee$  counterexamples  $= \emptyset$ 
30: Output:  $(V, \pi) \leftarrow$  certified certificate-controller pair

```

E Supplementary Experiments

E.1 Experiment Results

We re-ran our experiments with 5 trials. We report the average performance and standard deviation, visualized with error bars in the accompanying figures. Table 3 summarize the certified robustness across different methods in both environments. Figure 2 and Figure 3 illustrate the empirical robustness of each method under PGD attacks at various perturbations in both environments. We observe that the overall trends and conclusions remain consistent with Section 5.2 and Section 5.3.

E.2 Ablation Study

In Table 4 we summarize the effects of key training hyperparameters on robustness and convergence. For each setting, we report:

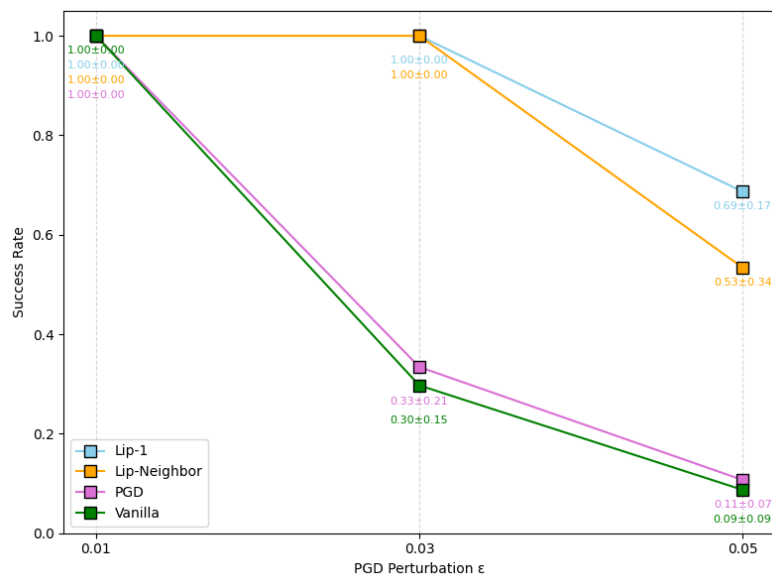


Figure 2: Success rate under various PGD perturbations in 2D docking

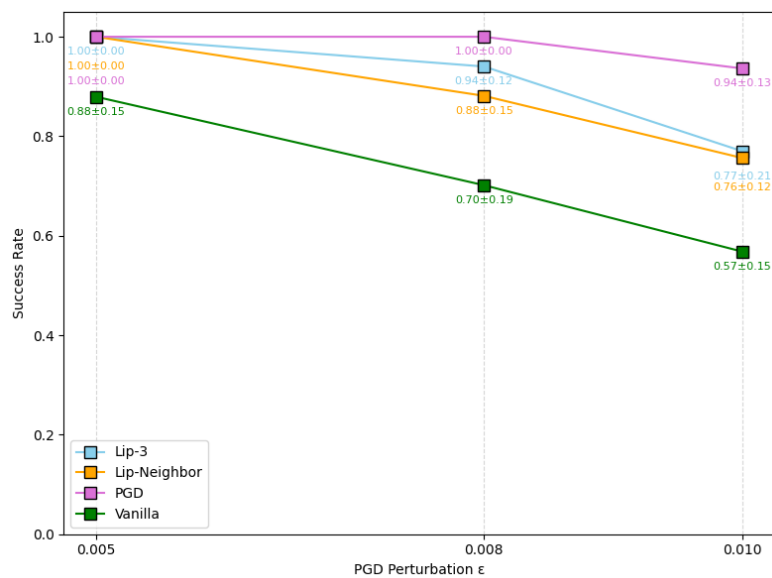


Figure 3: Success rate under various PGD perturbations in Inverted Pendulum

Method	Certified Bound (Mean $\pm$ Std)	Method	Certified Bound (Mean $\pm$ Std)
Vanilla	0.0042 $\pm$ 0.0012	Vanilla	0.0086 $\pm$ 0.0009
Lip-3	0.0072 $\pm$ 0.0017	Lip-1	0.0163 $\pm$ 0.0012
Lip-Neighbor	0.0062 $\pm$ 0.0022	Lip-Neighbor	0.0154 $\pm$ 0.0016
PGD	0.0081 $\pm$ 0.0004	PGD	0.0124 $\pm$ 0.0010

(a) Inverted Pendulum

(b) 2D Docking

Table 3: Certified perturbation bounds (mean $\pm$ std) for different training methods.

- the number of CEGIS loop iterations, which reflects the difficulty of achieving convergence;
- the certified perturbation bound, representing the model’s certifiable robustness;
- and empirical robustness, shown as the percentage of trajectories successfully reaching the goal under PGD attacks at different perturbation levels (0.01, 0.03, and 0.05).

Hyperparameter	Value	CEGIS Iterations	Certified Bound	0.01 adv	0.03 adv	0.05 adv
w	1000	4	0.014	100%	100%	23%
	500	5	0.013	100%	100%	35%
	100	4	0.013	100%	100%	45%
τ	0.8*	—	—	—	—	—
	1	3	0.015	100%	100%	85%
	2	6	0.013	100%	100%	45%
	3	6	0.013	100%	100%	54%
$\lambda_{\text{global-lip}}$	0.5	8	0.013	100%	83%	16%
	1	4	0.013	100%	100%	45%
	5	6	0.010	100%	100%	31%
	10	8	0.015	100%	100%	47%
	20	8	0.015	100%	100%	22%
ϵ	0.01	4	0.012	100%	100%	45%
	0.005	9	0.010	100%	83%	9.7%
	0.001	7	0.003	100%	10.9%	12.1%

Table 4: Effects of varying key hyperparameters on convergence and robustness. *Training with $\tau = 0.8$ did not converge within 12 hours.